[26-27_SEM1_IS451_A] - INFORMATION AND SYSTEMS SECURITY
DR. GOVINDHA YELURIPATI

ASSIGNMENT 2

GROUP MEMBERS
Edinam Afeafa Apawu Agyeman (72122027)
Petra Asima (42302027)
Josephine Forgive Doamekpor ($_{41892027}$)

Monday, October 19, 2026

1. Architecture

The portal is a Node.js and Express server running on localhost that makes all access decisions itself; the browser is treated as untrusted. Figure 1 shows the components.

The browser pages (enrol.html, login.html, student.html, lecturer.html) call the server over JSON. auth.js handles passkey registration and login using the SimpleWebAuthn library (v13). server.js handles sessions and access control. Both use helper functions in db/db.js, which run prepared SQL statements against a SQLite file (app.db).

app.db stores users and roles, one course and its scores, hashed one-time enrolment codes, passkey public keys, one-time challenges and hashed session tokens. No private key is stored on the server. The AES-GCM demonstration (crypto/aesgcm-demo.js) is a separate script using Node's built-in crypto module and does not touch the portal.

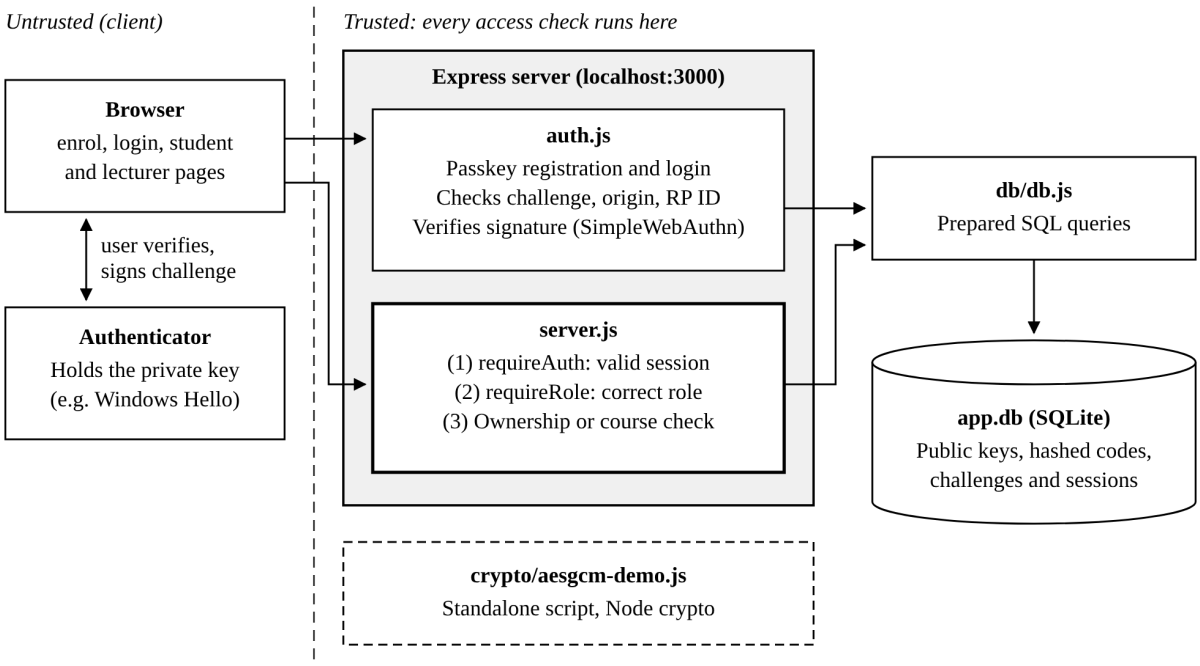


Figure 1: System architecture

The browser sends passkey ceremonies to auth.js and API requests with the session cookie to server.js, where checks 1 to 3 run in order before any data is read.

2. Role-based access control

Every rule in Table 1 is enforced in server.js, in three layers that run on each protected request.

1. **Session check (requireAuth).** The server reads the session cookie, hashes it and looks it up in the sessions table. A missing, expired, or revoked session gets a 401 "Please sign in".
2. **Role check (requireRole).** Lecturer-only routes (course roster, grade edits) return 403 to any other role.

3. **Ownership or assignment check, inside each route.** A student may read only the grades whose id equals the id in their own server-side session, never an id the browser supplies. A lecturer may access only the course whose lecturer_id is their own id, and only students enrolled in it.

The third layer is why two student accounts exist: S1 and S2 share a role, so a role check alone would let S1 read S2's grades. Inputs are validated (integer IDs, scores from 0 to 100) and all queries use prepared statements. Denials return a short generic message ("Access denied") with no record data. No endpoint creates users or changes roles; roles are set only by the seed script, which gives least privilege.

Table 1: Permission matrix

Resource or action	Student	Lecturer
View own profile and grades	Allow	Own profile: allow
View another student's grades	Deny	Allow only for assigned course
View assigned course roster	Deny	Allow
Edit course grades	Deny	Allow only for assigned course
Assign or change user roles	Deny	Deny

3. WebAuthn registration and login

A session is created only after the server has verified a real WebAuthn signature with the SimpleWebAuthn library. auth.js implements four endpoints under /api.

Registration (/register/options, then /register/verify)

1. The user enters a one-time enrolment code. The browser sends only that code, never a user id or role.
2. The server hashes the code and finds its owner in the database. Wrong, expired, and used codes all get the same 400 error.
3. The server generates a random challenge with generateRegistrationOptions and stores it with a 5-minute expiry. excludeCredentials stops the same authenticator registering twice for one user.
4. The browser calls startRegistration. The authenticator creates a new key pair after user verification (PIN or biometric). The private key stays on the device.
5. The server re-checks the code and consumes the challenge. It must be unused, unexpired and belong to that code's user.
6. verifyRegistrationResponse checks the challenge, the origin (http://localhost:3000) and the RP ID (localhost).

7. Only then does the server store the credential id, public key, counter and transports, and mark the code as used. No session is created here.

Login (/login/options, then /login/verify)

1. The server generates and stores a fresh authentication challenge (5-minute expiry).
2. The browser calls startAuthentication. The authenticator signs the challenge after user verification.
3. The server consumes the challenge, then looks up the credential id. An unknown credential gets 401 "Sign-in failed".
4. verifyAuthenticationResponse checks the signature against the stored public key, plus the challenge, origin, RP ID and counter. Any failure or thrown error gives 401 and no session.
5. On success the server updates the counter and creates a session: a random 256-bit token in an HttpOnly, SameSite=Strict cookie (30 minutes). Only the token's SHA-256 hash is stored.

Why is this secure? The server holds only public keys, so a database breach yields nothing that can sign in. The browser writes the real origin into the signed data and the server checks it, so an assertion captured by a phishing site on another origin fails. Challenges are random, single use and short lived, so a captured response cannot be replayed; consumeChallenge marks a challenge used with one conditional UPDATE, so two copies of the same response cannot both succeed. The signature counter can reveal a cloned authenticator if it goes backwards, though synced passkeys often report 0.

Enrolment is a demonstration bootstrap. The one-time codes printed by the setup script stand in for identity proofing. A real university would verify identity in person or through an existing trusted system before issuing a code, deliver it over a trusted channel, and offer a supervised recovery process.

4. AES-GCM demonstration

The script decrypts the untouched ciphertext correctly and rejects it after one byte is changed. crypto/aesgcm-demo.js uses Node's built-in crypto module to encrypt "Student consultation note – demonstration data only" with AES-256-GCM. Each run uses a newly generated random 256-bit key and a new random 12-byte nonce. It then flips one bit in the first ciphertext byte, and decryption fails with "Unsupported state or unable to authenticate data".

Public-key authentication (WebAuthn). The authenticator signs a fresh server challenge with a private key that never leaves the device. The server verifies the signature with the stored public key, which proves possession of the private key without revealing it.

Confidentiality and integrity. AES in counter mode turns the note into ciphertext that cannot be read without the key. GCM also computes a 128-bit authentication tag over the ciphertext. Decryption checks the tag before returning any plaintext, so any change to the ciphertext, tag or nonce is detected.

Nonces must not repeat with the same key. Counter mode makes a keystream from the key and nonce. Reusing both means two messages share a keystream, so XOR-ing the two ciphertexts

reveals the XOR of the plaintexts. Reuse also leaks the authentication key, letting an attacker forge valid tags. The script generates a fresh random nonce for every encryption.

Key protection.

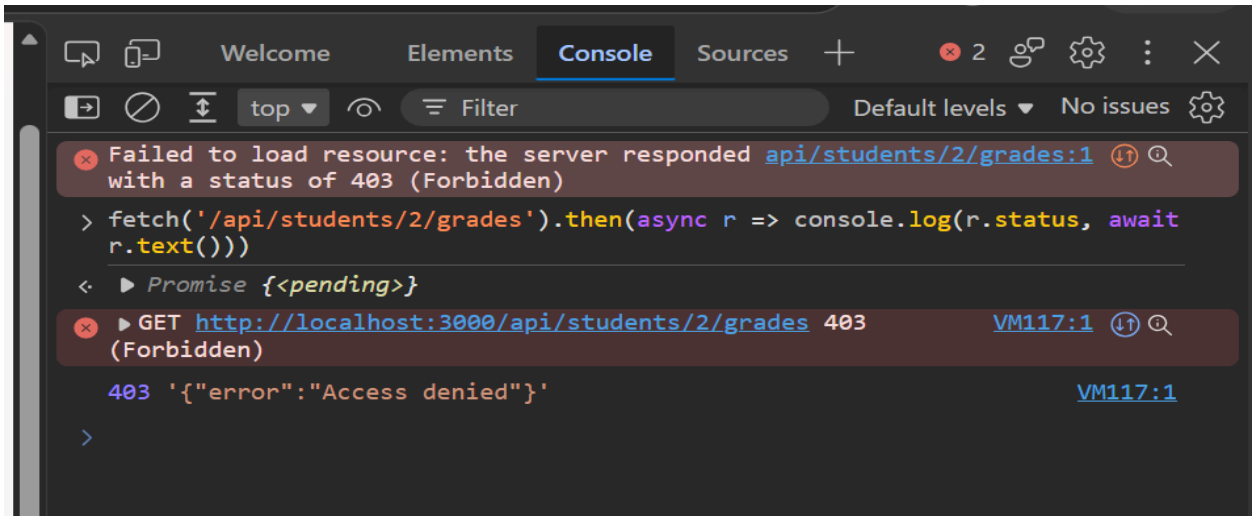
Keys come from a secure random generator and are never hard-coded or committed to Git. In a real system they would be held in a key management service or protected environment settings, with limited access, rotation, and storage separate from the encrypted data. The script prints the key only because this is a classroom demonstration.

5.

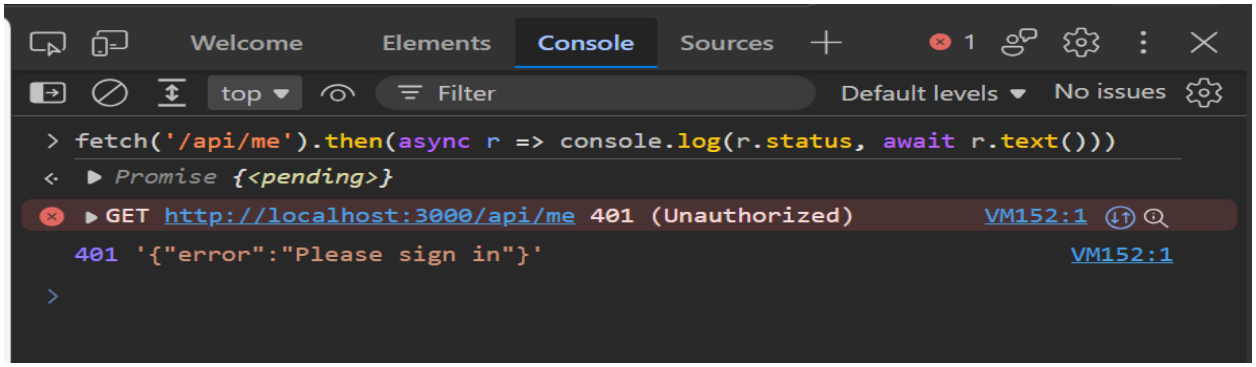
Table 2: Test results

Test	What we did	What happened	Evidence
T1	Registered a passkey for S1 with its one-time code, then signed in	Registration and server-verified login succeed	T1a, T1b, T1c
T2	Signed in as S1 and viewed the dashboard	Allowed	T2
T3	As S1, requested S2's grades by button and by direct URL (/api/students/2/grades)	Denied, no S2 data	T3
T4	As S1, sent PUT /api/courses/1/grades/1 with score 100	Denied on the server	T4
T5	Signed in as L1, changed S1's score, refreshed	Allowed; change persists	T5a, T5b
T6	Called /api/me signed out, then again after logout	Denied in both cases	T6a, T6b
T7	Sent a login with an unregistered credential id to /api/login/verify	Authentication denied; no session	T7a, T7b
T8	Ran crypto/aesgcm-demo.js: encrypt, decrypt, alter one byte, decrypt again	Plaintext only when unchanged	T8

T3 denial in the console



T6 401 after logout



T8 tamper rejection

```
PS C:\Users\Edinam\InfoSec_Assignment2> node crypto/aesgcm-demo.js
Original note : Student consultation note – demonstration data only
Key (hex)     : 9fce28ba88d5b3408284a5c2208bb40abc42968bb598f6bd9837aff3c49800
f9
IV (hex)      : a925f7a78b324892de91d64b
Ciphertext    : 025e7b7d438709a2491bd002bd9fc832acdba08976305fde3f7d4e6db2b217
dd9b582ee9bd7b4e2e8449fe5204d50ca444f5b03e73

1) Decrypting valid ciphertext...
SUCCESS: Student consultation note – demonstration data only

7b4e2e8449fe5204d50ca444f5b03e73
TAMPER DETECTED: decryption rejected (Unsupported state or unable to authentic
ate data)
```

6. Residual risks and limitations

The prototype handles authentication and authorisation correctly on localhost, but three gaps would need closing before real use.

Table 3: Residual risks

Risk	What the prototype does	What a real deployment would still need
Insecure initial account provisioning	Codes are random, stored only as SHA-256 hashes, single-use, and expire after 7 days. The user and role come from the database, not the browser. But the setup script prints codes to a console; the codes are only 8 hex characters, and there is no limit on guessing attempts.	In-person or institutional identity proofing before issuing codes, longer codes, rate limiting and lockout, delivery over a trusted channel.
Lost device and account recovery	None. A user who loses their only authenticator is locked out, and a new code needs the setup script, which rebuilds the whole database.	Several passkeys per user, a supervised recovery process with identity checks, and a way to revoke a lost passkey.
Compromised session and weak audit logging	Session tokens are random 256-bit values stored only as hashes, in HttpOnly, SameSite=Strict cookies. Sessions expire after 30 minutes and logout revokes them on the server. But there is no HTTPS (localhost only), no Secure cookie flag, no re-authentication before grade changes, and no audit log.	HTTPS with Secure cookies, re-authentication for sensitive actions such as grade changes, and tamper-resistant audit logs with alerts.

Individual contribution statements

Edinam, 72122027

Edinam set up the GitHub repository and built the database layer: `db/schema.sql`, `db/seed.js` and `db/db.js`, including hashed session tokens and the single-use challenge function. I wrote `server.js`, which enforces the session check (`requireAuth`), the role check (`requireRole`) and the ownership and course-assignment checks on every protected route. She also ran tests T2 and T5, saved their evidence, and wrote the report's access-control section.

Petra Asima, 42302027

Petra built the passkey part of the portal: `auth.js`, with the four WebAuthn endpoints for registration and login using SimpleWebAuthn, and the `enrol.html` and `login.html` pages. She also wrote the AES-GCM demonstration in `crypto/aesgcm-demo.js`. She ran tests

T1, T7, and T8, saved their evidence, and wrote the architecture, passkey, and AES-GCM sections of the report.

Josephine Doamekpor, 41892027

Josephine built the student and lecturer dashboard pages (`student.html` and `lecturer.html`), including the buttons that attempt forbidden actions for the denial tests. She wrote the README, covering setup, test accounts, safe passkey enrolment, and how to repeat T1 to T8. I ran tests T3, T4, and T6, saved the evidence, wrote the test results and residual risks sections, merged the report, and recorded the demonstration.

References

MDN Web Docs. (n.d.-a). *SubtleCrypto: encrypt() method*. Mozilla. Retrieved October 19, 2026, from https://developer.mozilla.org/en-US/docs/Web/API/SubtleCrypto/encrypt

MDN Web Docs. (n.d.-b). *Web Authentication API*. Mozilla. Retrieved October 19, 2026, from https://developer.mozilla.org/en-US/docs/Web/API/Web_Authentication_API

Node.js. (n.d.). *Crypto*. Node.js documentation. Retrieved October 19, 2026, from https://nodejs.org/api/crypto.html

OWASP Foundation. (n.d.-a). *Authorization cheat sheet*. OWASP Cheat Sheet Series. Retrieved October 19, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

OWASP Foundation. (n.d.-b). *Cryptographic storage cheat sheet*. OWASP Cheat Sheet Series. Retrieved October 19, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html

SimpleWebAuthn. (n.d.). *SimpleWebAuthn documentation* (Version 13). Retrieved October 19, 2026, from https://simplewebauthn.dev/docs/